

Perancangan Knight-Chaos Cipher: Algoritma Kriptografi Berbasis Langkah Kuda Catur dan Chaotic Map

Nama: Muhammad Rizky Hajar

NIM: 24.55.2714

Program Studi: PJJ Informatika

Konsentrasi: Big Data dan Predictive Analytics (BDPA)

Mata Kuliah: Cyber Security

1. Pendahuluan

Kriptografi merupakan salah satu komponen utama dalam keamanan siber karena digunakan untuk menjaga kerahasiaan, integritas, dan autentikasi data. Algoritma modern menggunakan prinsip substitusi dan permutasi berulang untuk mengubah plaintext menjadi ciphertext yang sulit dianalisis. Pada tugas ini dirancang sebuah algoritma kriptografi baru dengan nama **Knight-Chaos Cipher (KCC-128)**.

Rancangan KCC-128 menampilkan kreativitas pada modifikasi komponen utama cipher. Perbedaan utama KCC-128 adalah penggunaan **logika permainan catur**, khususnya langkah kuda, sebagai mekanisme permutasi internal, serta penggunaan **logistic chaotic map** sebagai pembentuk S-Box dinamis. Modifikasi diterapkan pada arsitektur substitusi, permutasi, difusi, dan key schedule.

2. Konsep Dasar Algoritma

KCC-128 bekerja sebagai block cipher dengan ukuran blok 128 bit atau 16 byte. Setiap blok dipandang sebagai matriks 4x4. Algoritma menggunakan 10 round, dan setiap round memiliki material kunci yang berbeda. Material tersebut terdiri dari round key, S-Box dinamis, inverse S-Box, jalur permutasi langkah kuda, inverse permutation, dan pola rotasi bit.

Kunci pengguna diproses dengan SHA-256 untuk menghasilkan master seed. Master seed ini kemudian digunakan untuk membentuk round material. Key schedule menghasilkan byte untuk operasi XOR sekaligus memengaruhi bentuk internal cipher, yaitu S-Box, rute permutasi, rotasi bit, dan difusi byte. Hal ini membuat struktur transformasi bergantung pada kunci.

3. Arsitektur Knight-Chaos Cipher

3.1 Dynamic S-Box Berbasis Chaotic Map

S-Box pada KCC-128 tidak bersifat tetap. Setiap round menghasilkan S-Box berbeda dengan menggunakan logistic chaotic map:

$$x(n+1) = r * x(n) * (1 - x(n))$$

Nilai awal x dan parameter r diturunkan dari round seed. Deret chaotic yang dihasilkan digunakan untuk memberi skor pada nilai byte 0 sampai 255, kemudian byte diurutkan

berdasarkan skor tersebut. Hasil akhirnya adalah permutasi 0..255 yang dapat digunakan sebagai S-Box. Karena S-Box adalah permutasi, inverse S-Box dapat dibuat untuk kebutuhan dekripsi.

3.2 Permutasi Langkah Kuda Catur

Setiap blok 16 byte dipetakan ke papan 4x4. Posisi byte kemudian dipermutasi mengikuti pola langkah kuda catur. Titik awal ditentukan oleh round seed. Jika terdapat lebih dari satu langkah valid, algoritma memilih langkah dengan prinsip jumlah langkah lanjutan terkecil, lalu menggunakan round seed untuk memecahkan tie. Mekanisme ini mengintegrasikan logika game secara langsung ke komponen permutasi cipher.

Berbeda dari pergeseran linear, langkah kuda menghasilkan perpindahan tidak berurutan antara baris dan kolom. Hal ini membantu menyebarkan posisi byte plaintext ke posisi ciphertext yang lebih sulit ditebak.

3.3 BitShiftMix dan Byte Diffusion

Setelah substitusi dan permutasi, setiap byte dirotasi berdasarkan nilai round key. Kemudian dilakukan byte diffusion dan Feistel-like mixing. Lapisan ini penting karena substitusi dan permutasi saja belum cukup kuat untuk menyebarkan perubahan bit. Byte diffusion membuat satu byte memengaruhi byte lain, sedangkan Feistel-like mixing menjaga proses tetap invertibel sehingga dekripsi dapat dilakukan dengan benar.

3.4 Mode Chaining

Untuk pesan yang lebih panjang dari satu blok, KCC-128 menggunakan chaining internal. Blok plaintext pertama di-XOR dengan IV yang diturunkan dari kunci, sedangkan blok berikutnya di-XOR dengan ciphertext blok sebelumnya. Tujuannya adalah mencegah blok plaintext yang sama menghasilkan pola ciphertext yang sama pada posisi berbeda.

4. Proses Enkripsi dan Dekripsi

Pada proses enkripsi, plaintext diberi padding PKCS#7 agar panjangnya kelipatan 16 byte. Setiap blok kemudian melewati 10 round. Urutan transformasi pada setiap round adalah AddRoundKey, ChaoticSubBytes, KnightPermutation, BitShiftMix, ByteDiffusion, dan FeistelMix.

Proses dekripsi menjalankan inverse dari seluruh transformasi dalam urutan terbalik. Karena S-Box, permutasi, rotasi bit, byte diffusion, dan Feistel-like mixing semuanya dirancang invertibel, ciphertext dapat dikembalikan menjadi plaintext tanpa korupsi.

Secara ringkas, rancangan komponen KCC-128 dapat dilihat pada tabel berikut.

Komponen	Fungsi	Sumber Keunikan
AddRoundKey	Mencampur state dengan round key	Round key diturunkan dari SHA-256 kunci pengguna
ChaoticSubBytes	Mengganti nilai byte	S-Box berubah per round dari logistic map
KnightPermutation	Menukar posisi byte	Jalur langkah kuda catur pada

Komponen	Fungsi	Sumber Keunikan
BitShiftMix	Memutar bit dalam byte	papan 4x4 Jumlah rotasi ditentukan round seed
ByteDiffusion	Menyebarkan pengaruh antarbyte	Setiap byte dipengaruhi carry dan tetangga
FeistelMix	Memperkuat avalanche effect	Mixing invertibel sehingga dekripsi tetap mungkin

Pseudocode inti enkripsi satu blok adalah sebagai berikut.

```

state <- block
FOR each round material DO
    state <- state XOR roundKey
    state <- SBox[state]
    state <- KnightPermutation(state)
    state <- RotateLeft(state)
    state <- ByteDiffusion(state)
    state <- FeistelMix(state)
END FOR
RETURN state

```

Pada dekripsi, urutan tersebut dibalik sepenuhnya. Bagian ini penting karena algoritma kriptografi tidak cukup hanya menghasilkan ciphertext acak; algoritma juga harus mampu mengembalikan data asli secara deterministik ketika kunci yang sama digunakan.

5. Metodologi Pengujian

Pengujian dilakukan dengan tiga tujuan. Pertama, menguji kebenaran fungsional, yaitu memastikan ciphertext yang didekripsi kembali identik dengan plaintext awal. Kedua, menguji entropy untuk melihat apakah distribusi byte ciphertext lebih acak dibanding plaintext. Ketiga, menguji avalanche effect dengan mengubah satu bit pada plaintext dan menghitung persentase bit ciphertext yang berubah.

Rumus entropy Shannon yang digunakan adalah:

$$H(X) = - \sum p(x) \log_2 p(x)$$

Jika ciphertext memiliki distribusi byte yang baik, nilai entropy akan bergerak mendekati 8 bit per byte. Untuk avalanche effect, nilai ideal secara konseptual mendekati 50%, karena perubahan kecil pada input seharusnya mengubah sekitar separuh bit output.

Selain demo CLI, dibuat juga demo interaktif berbasis web. Demo ini menampilkan state 4x4 dari blok pertama, sehingga proses transformasi dapat diamati secara bertahap. Setiap langkah round dapat dilihat satu per satu melalui tombol Prev, Next, dan Play.

6. Hasil Implementasi dan Sample Run

Implementasi dilakukan menggunakan Python. Demo menunjukkan bahwa plaintext berhasil dienkripsi menjadi ciphertext dalam format hex dan base64, kemudian didekripsi kembali menjadi plaintext awal. Hasil sample run menunjukkan:

- Decryption OK: True
- Entropy plaintext: sekitar 4.3846 bit/byte
- Entropy ciphertext: sekitar 6.6660 bit/byte
- Avalanche effect: sekitar 48.34%

Nilai avalanche effect tersebut menunjukkan bahwa perubahan satu bit pada plaintext menghasilkan perubahan besar pada ciphertext. Nilai ini mendekati karakteristik difusi yang diharapkan pada cipher modern, meskipun pengujian ini masih bersifat sederhana dan terbatas.

Demo interaktif juga berhasil dijalankan. Pada halaman web, plaintext dan key dapat diubah, kemudian pengguna dapat menekan tombol **Encrypt & Animate**. Sistem menampilkan ciphertext dalam bentuk hex dan status dekripsi. Jika algoritma berjalan benar, halaman menampilkan pesan bahwa plaintext kembali utuh.

7. Analisis Keamanan

Analisis entropy menunjukkan bahwa ciphertext memiliki distribusi byte yang lebih acak dibanding plaintext. Plaintext berupa kalimat natural memiliki distribusi karakter yang cenderung berulang, sedangkan ciphertext menghasilkan byte yang lebih tersebar.

Avalanche effect diuji dengan mengubah satu bit pertama pada plaintext dan membandingkan ciphertext sebelum dan sesudah perubahan. Hasil sekitar 48.34% menunjukkan bahwa hampir separuh bit ciphertext berubah. Ini merupakan indikator bahwa kombinasi S-Box dinamis, permutasi langkah kuda, rotasi bit, difusi byte, Feistel-like mixing, dan chaining berhasil menciptakan penyebaran perubahan yang cukup baik.

Analisis frekuensi juga menunjukkan bahwa pola karakter plaintext tersamarkan pada ciphertext. Byte yang paling sering muncul pada ciphertext memiliki frekuensi rendah dan tersebar. Pengujian lanjutan dapat dikembangkan melalui differential cryptanalysis, linear cryptanalysis, uji randomness yang lebih besar, dan evaluasi terhadap skenario chosen-plaintext.

Jika dibandingkan dengan cipher klasik seperti Caesar atau Vigenere, KCC-128 memiliki ruang transformasi yang lebih kompleks karena bekerja pada byte dan memakai beberapa lapisan round. Cipher klasik umumnya rentan terhadap analisis frekuensi karena hubungan antara plaintext dan ciphertext masih relatif langsung. Pada KCC-128, hubungan tersebut diputus melalui S-Box dinamis, permutasi langkah kuda, dan difusi antarbyte.

KCC-128 mengambil inspirasi umum dari prinsip substitution-permutation network dan membangun komponen khas melalui S-Box chaotic serta permutasi berbasis mekanisme game. Kontribusi tugas ini terletak pada kreativitas arsitektur yang menggabungkan konsep catur, chaotic map, dan difusi invertibel.

8. Kelebihan dan Keterbatasan

Kelebihan KCC-128 adalah adanya modifikasi nyata pada komponen inti cipher. Algoritma ini menggabungkan mekanisme game dan konsep matematika non-konvensional dalam satu arsitektur. S-Box tidak tetap, permutasi tidak linear, dan key schedule memengaruhi struktur transformasi.

Pengembangan berikutnya dapat diarahkan pada pengujian statistik yang lebih luas, variasi IV acak, dan perbandingan hasil avalanche pada dataset plaintext yang lebih besar. Penggunaan SHA-256 dalam key schedule membantu determinisme dan distribusi seed, sementara struktur round memberi ruang eksplorasi untuk optimasi lanjutan.

Pengujian saat ini menggunakan sampel demonstrasi. Nilai entropy dan avalanche effect dapat diperluas dengan ukuran data uji yang lebih besar. Mode IV pada implementasi ini diturunkan secara deterministik dari kunci untuk konsistensi demo; pengembangan berikutnya dapat memakai IV acak yang disimpan bersama ciphertext.

9. Kesimpulan

Knight-Chaos Cipher berhasil dirancang sebagai algoritma kriptografi yang memodifikasi arsitektur cipher pada level substitusi, permutasi, difusi, dan key schedule. Integrasi langkah kuda catur memberikan pola permutasi berbasis game, sedangkan logistic chaotic map menghasilkan S-Box dinamis berbasis konsep matematika. Implementasi Python menunjukkan bahwa enkripsi dan dekripsi berjalan benar, serta analisis awal menunjukkan entropy ciphertext meningkat dan avalanche effect cukup kuat.

KCC-128 memenuhi tujuan tugas untuk merancang algoritma kriptografi baru yang kreatif, dapat diimplementasikan, dapat didemonstrasikan, dan dapat dianalisis melalui entropy, avalanche effect, serta frekuensi byte.

Referensi

1. William Stallings, *Cryptography and Network Security: Principles and Practice*.
2. Joan Daemen dan Vincent Rijmen, *The Design of Rijndael: AES - The Advanced Encryption Standard*.
3. Bruce Schneier, *Applied Cryptography*.
4. Robert L. Devaney, *An Introduction to Chaotic Dynamical Systems*.